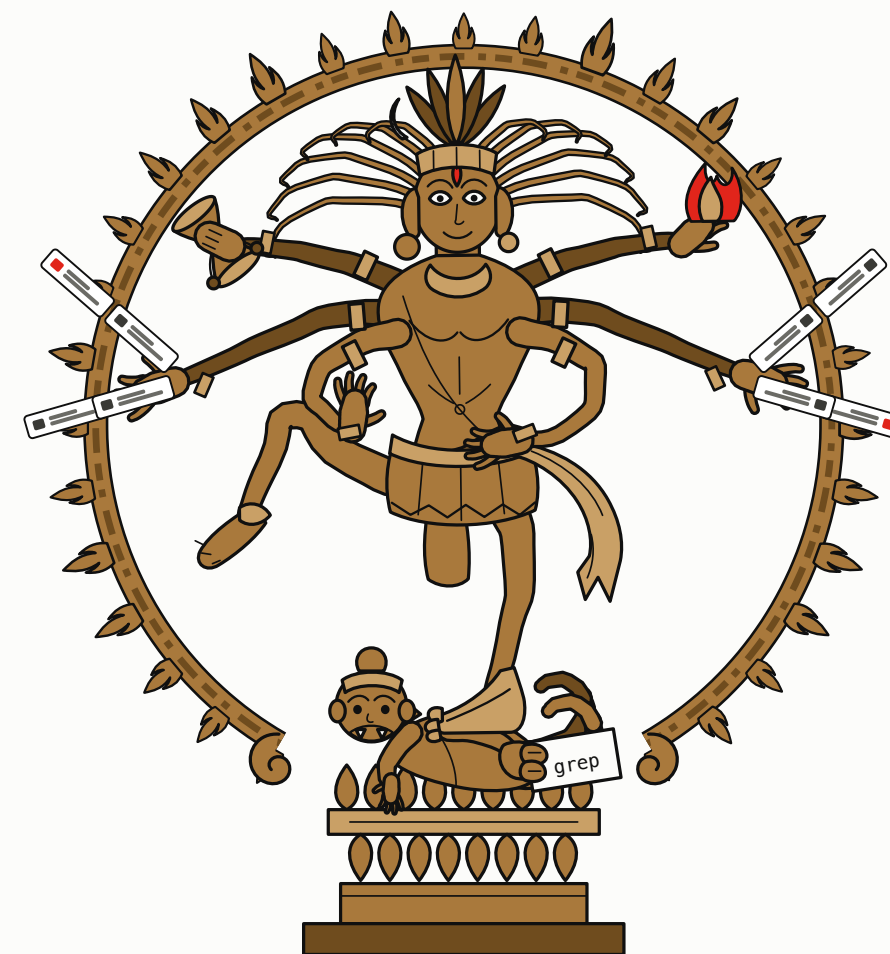




ALICE

WEDNESDAY 19 AUGUST 2026

Modern Logging Platform with **Machine Learning** for HPC



```
$ tail -f /dev/cosmos █
```

PRESENTED BY

Marko Sladojevic

SUPERVISORS

Lubos Krcal
Federico Ronchetti

01 / 16

Outline

What logging looks like today, what we suggest instead, and what is still missing.



03–06

The problem, and the design

Logging today · What we want to build · Our suggested architecture · Federated search



07–11

Choosing the tech

Why Fluent Bit · Why OpenSearch · Inside the collector · How we use OpenSearch · Deployment with Ansible



12–14

What people see, and what watches it

The interface · Log anomaly detection · Anomaly detection on metrics



15

Coming soon

What is coming soon

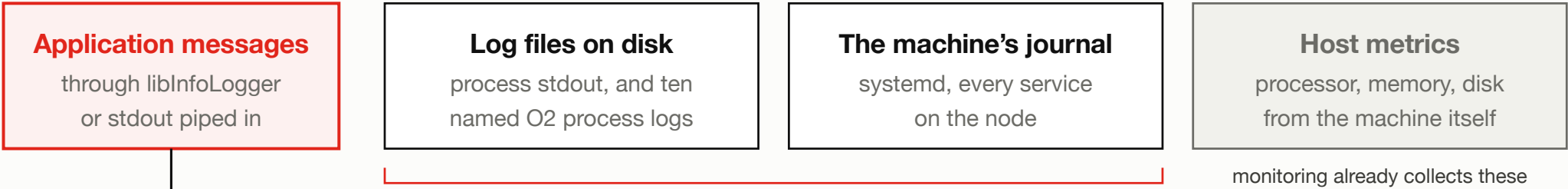
Logging today

InfoLogger moves the messages a process sends it. That is one road. Most of what a node writes never takes it.

SHIFTERS READ THE LOGS BY HAND

01 WHAT A NODE WRITES

InfoLogger carries what a process sends it, and nothing else.

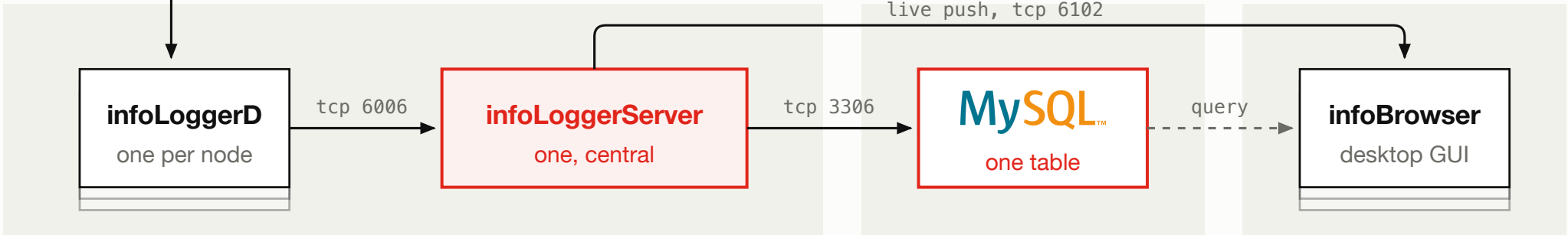


Neither of these reaches InfoLogger

02 COLLECT

03 STORE

04 DISPLAY



One process and one table for the whole farm

05 THE LIMITS TODAY

• ONE SQL TABLE

Every message in the farm lands
in one messages table.

• NO AGGREGATION

infoBrowser lists single lines.
It draws no counts over time.

• ONE POLICY, ONE COPY

One retention rule covers every
message. Nothing is replicated.

What we want to build

Six objectives shape the design that follows. Each one names a property the platform must have. Together they keep the volume where it is made, and the value where it is safe.



NO BULK OVER THE WIRE

Application logs at info severity are most of the volume. Each worker indexes its own on its own disk. They don't cross the network.



DISTRIBUTED

Five machines share the work as one cluster. Together they handle more throughput than any one machine could.



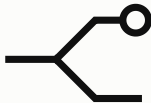
DURABLE

Some logs matter more than others. We need a storage tier that can ensure higher replication and retention policies for them.



FAULT TOLERANT

Two of the three storage machines are enough to keep going. One can die and we lose nothing. In production the tier is bigger, so more can die. Lose a worker and we lose only its local logs.



SPLIT BY SEVERITY

We split some log families by severity, not by source. That puts the volume on one side and the value on the other. Each kind of log then gets its own keep time and its own number of copies.



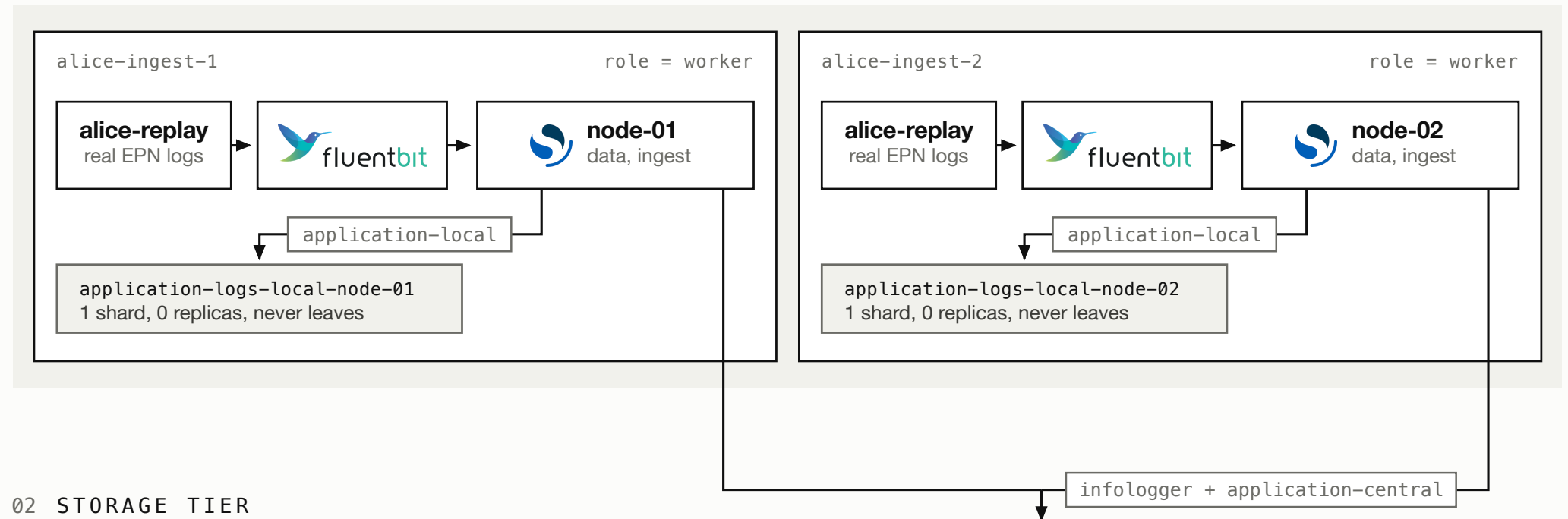
WORKERS STAY CHEAP

A worker indexes its own logs and answers a query now and then. It never manages the cluster. It takes four of the 128 cores an EPN has, and little memory. We must not disturb the expensive timeframe reconstruction.

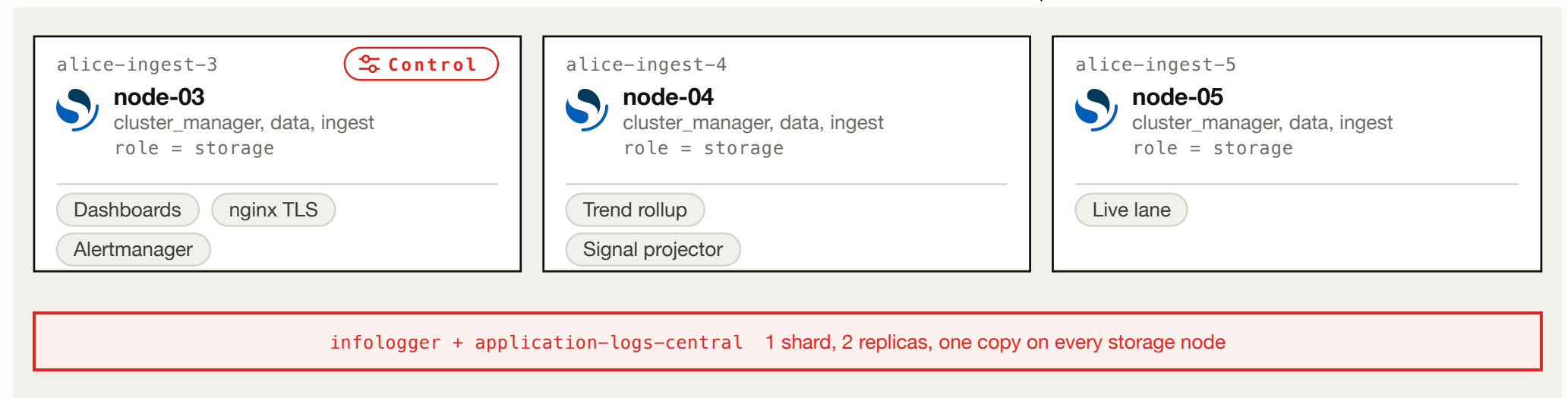
Our suggested architecture

5 VMS – 2 WORKER, 3 STORAGE
ONE OPENSEARCH CLUSTER
NATIVE SYSTEMD, NO DOCKER

01 WORKER TIER



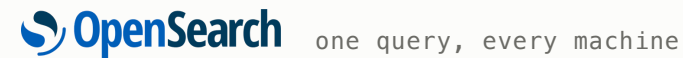
02 STORAGE TIER



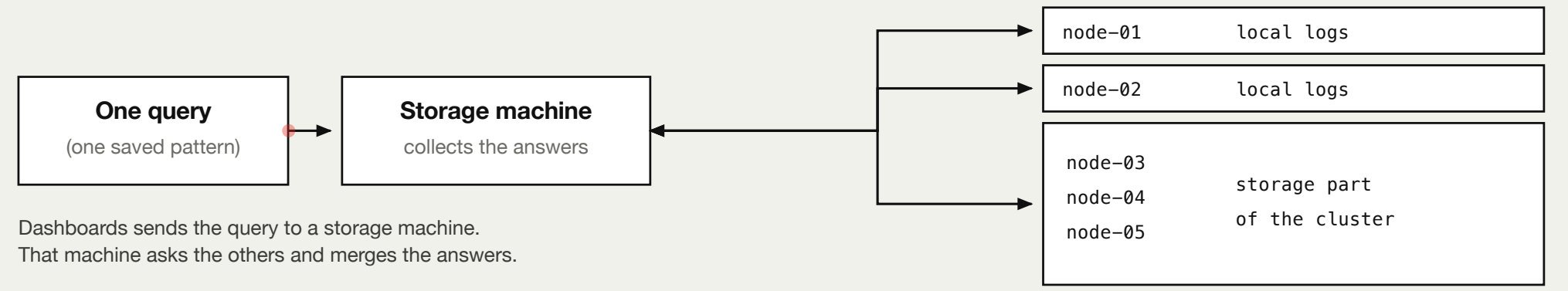
Federated search

Every machine writes its own logs and keeps them. So the real question is how one person searches all of them at once. Our answer: make the machines one cluster, and ask it once.

ONE CLUSTER, FIVE MACHINES
ONE UPGRADE WINDOW FOR ALL
PUSH UP, NEVER POLL DOWN



01 ASK ONCE – EVERY NODE ANSWERS



02 ONE CLUSTER, NOT MANY



We run one cluster. The other way is many clusters, joined together by cross-cluster search.

PROS OF MANY CLUSTERS

- + One bad query cannot slow the other clusters.
- + Each cluster upgrades on its own schedule.

PROS OF ONE CLUSTER

- + Built-in cross-cluster search does not scale to 100 machines.
- + The cluster places shards and replicas itself, across all machines.
- + One set of users, roles and saved searches.
- + Detection and alerting streamlined.
- + No federation layer of our own to write and keep alive.

04 FANOUT – NOT EVERYTHING IS PERFECT

Every machine pays

One search asks all of them, so all of them spend processor time on it. It is as slow as the slowest machine.

Reconstruction comes first

Those same machines reconstruct events. That is their real job, and log search must not compete with it.

So we ask only when we must

The live lane covers watching. But “which machines saw this error in run 561234?” has to ask all of them.

Why Fluent Bit

A lightweight forwarder runs on the edge beside the program that writes the logs. It reads each line, decides where the line belongs and sends it on. It stores nothing.

4 CANDIDATES WEIGHED
MATURITY AND SIZE DECIDED IT
A QUEUE IS A DIFFERENT JOB

01 THE CANDIDATES

Three are Apache 2.0, Telegraf is MIT. Job and governance are what differ.

	LANGUAGE	VENDOR’S MEMORY FIGURE	WHO GOVERNS IT	
	C	about 450 KB	CNCF*, graduated	CHOSEN
Where the community has converged. Its own project page counts fifteen billion deployments, and it has been in production for a decade. CNCF* graduated: the roadmap sits in a foundation, not in one company.				
	C and Ruby	over 60 MB	CNCF, graduated	
The same project one size up, and the older of the two. A Ruby package tree on every worker is what we did not want.				
	Rust	no vendor figure	Datadog	
Small and fast, and the benchmarks disagree on which of the two is smaller. One company decides its future.				
	Go	no vendor figure	InfluxData	A DIFFERENT JOB
A metrics agent, and a good one. ALICE O2 already runs Telegraf to gather machine and service metrics. It emits measurements in line protocol: a name, tags, numbers, a clock. That shape is a metric, not a log line.				

02 WHY NOT A MESSAGE QUEUE

 	Neither can tail a file, parse a line or stamp a clock. A forwarder still runs in front of the queue. The queue itself costs three more machines and a quorum.
A broker moves records. It does not make them.	

* CNCF is the Cloud Native Computing Foundation, the Linux Foundation body that also holds Kubernetes and Prometheus. Graduated is its highest maturity level. The 450 KB is the vendor’s figure for an idle pipeline; ours peaked between 133 and 228 MB under load, measured on the collector slide.

Why OpenSearch

We need a search engine that indexes every field and the message body, counts them over time, and brings anomaly detection with it. Four candidates were weighed. Only one does all three.

4 CANDIDATES WEIGHED
AN OPEN LICENCE WAS THE FLOOR
DETECTION DECIDED IT

01 THE CANDIDATES

	LICENCE AND GOVERNANCE	WHAT IT CAN ANSWER	WHAT IT COSTS US TO RUN	
	Apache 2.0, Linux Foundation	Every field and the message	Detection included, free	CHOSEN
The only candidate that is fully open, indexes the message body, and brings anomaly detection and alerting in the box. The detection slides that follow run on it. CERN IT's own monitoring service runs it too, so this is not a novel bet.				
	Three licences, one company	Every field and the message	Detection is a paid tier	
Anomaly detection and alerting sit in Elastic's paid tiers, so the detection half of this deck becomes a purchase.				
	Apache 2.0, one company	Every field and the message	We build the detection	
It wins on disk and loses on machinery. Choosing it means we run 17 detectors and 28 alerting rules ourselves.				
	AGPLv3, one company	Labels only, not the message	No detection to host	
It indexes labels, not the text, so counting errors per host is a scan. Its guidance forbids 211 hostnames as labels.				

02 WHAT WE GET THAT WE DO NOT HAVE TODAY

 Search the text any word, over a bunch of logs	 Filter any mapped field host, detector, run, partition	 Count over time errors per host, per minute
 Detect and alert included, and already running	 Replicate every record three copies, so a machine may die	 Shard the index writes spread, so throughput grows



Inside the collector

One process on every worker. Every line takes the same five steps, and the whole job is decided before it leaves the machine.

MEMORY ENVELOPE

64 chunks in memory	128 MB
Documented worst case	307 MB
Measured, steady load	133–228 MB
Measured, sink outage	404 MB



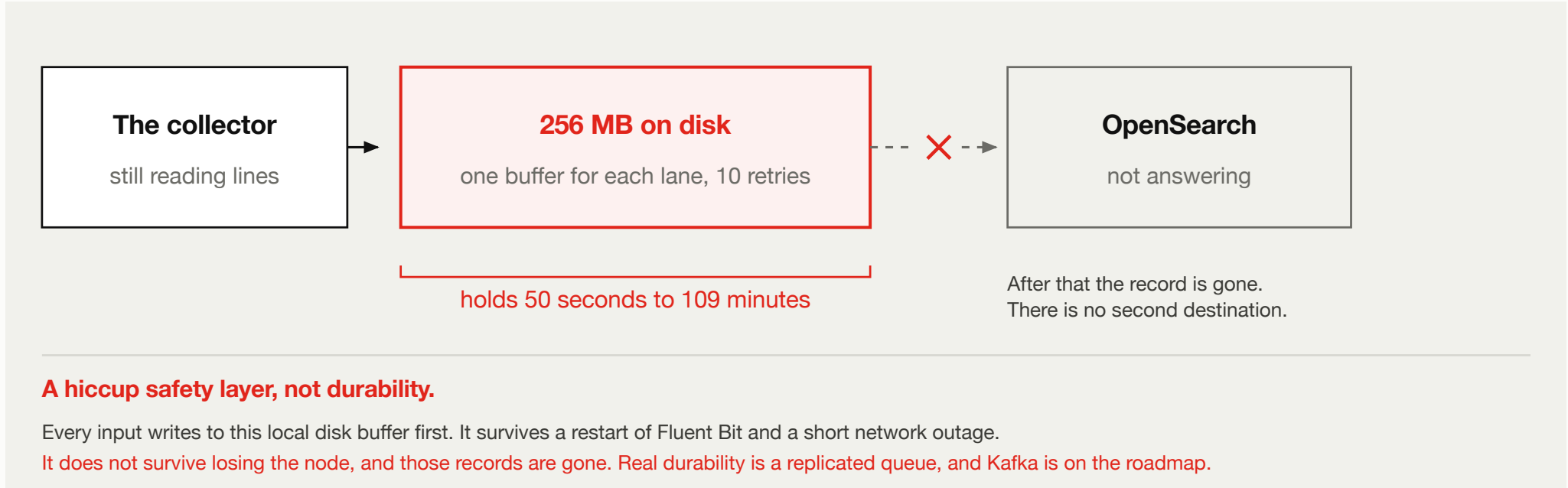
one process on every worker VM

4 inputs · 4 tags · 5 outputs

01 WHAT IT DOES TO A LINE



02 WHAT THE DISK BUFFER REALLY IS



How we use OpenSearch

One cluster, split in two. The bulk of the logs never leaves the machine that wrote them. The rest is copied onto three machines.

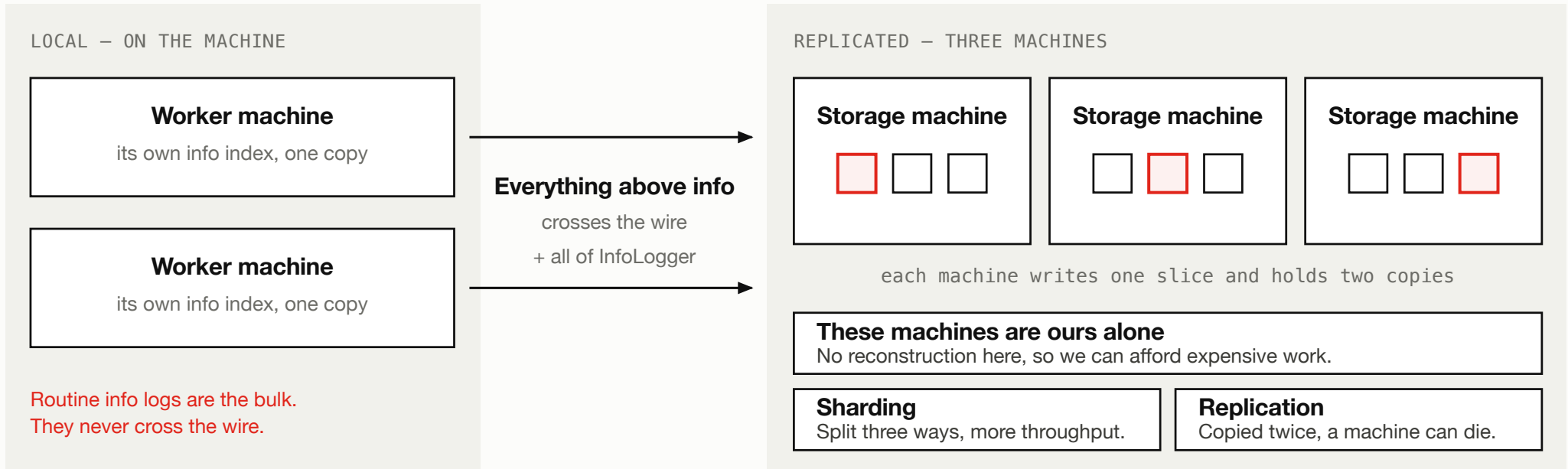
ONE CLUSTER, FIVE MACHINES
THREE COPIES OF THE VALUABLE HALF
ZERO COPIES OF THE CHEAP HALF



one cluster, five machines, two halves

All five machines hold data. The three storage machines run the cluster.

01 THE SPLIT, AND WHY IT IS A SPLIT



One storage machine may die and we lose nothing.

Every record is on all three. The cluster needs two of the three to agree, so it keeps running.

The local half is one copy on purpose. If a worker dies you lose that machine's routine logs, which is the cheap half.

02 THREE THINGS TO WATCH



JVM heap

The memory the Java process may hold. Every index costs heap, so we cap how many exist.



Memory outside the heap

The search index and the file cache use it. We lock the memory down, so a node can never swap.



ISM — Index State Management

Indices roll on a clock, old ones are deleted. Routine logs 8 days, others 35, InfoLogger 56.



Deployment with Ansible

Ansible won because it leaves nothing running on the five machines. It acts only when a person starts it. The same tool also creates the machines.

4 TOOLS WEIGHED
THE AGENT DECIDED IT
PLACEMENT IS DATA, NOT CODE

01 THE CANDIDATES

An agent is a program the tool installs and leaves running on every machine.

AGENT ON EVERY MACHINE		WHEN IT RUNS	CREATES THE MACHINES	
ANSIBLE		None, over SSH	When we run it	Yes, same tool
A push over SSH puts nothing new on a CERN host. One tool creates the five machines and configures them.				
puppet with Foreman		Agent and certificate	On a timer	Yes, with Foreman
The CERN standard, and it repairs drift by itself. It wants an agent and a signed certificate on every node.				
SALT PROJECT and Chef		Agent and a server	On demand or a timer	No
Salt fans out fastest, but only at farm scale. Chef needs a second language. Both put an agent on every node.				
HashiCorp Terraform		None	When we run it	Yes, and nothing else
It builds machines well, and its vendor calls configuring them a separate job. That is two tools, not one.				

02 OUR DESIGN DECISIONS

17 roles · 7 inventory groups · 5 machines

	Placement is data	One inventory file says where each service runs. Moving one service to another machine changed no code.
	Every step converges	A failed run is simply run again. Machines, monitors and detectors are created by name, never twice.
	Gates stop the run	12 checks fail the deploy rather than report success. The last proves 28 monitors and 17 detectors exist.
	Secrets stay encrypted	Every secret lives in one encrypted file in the repository. The cloud credential never touches disk.

ALICE build machines already run Puppet under the CERN Foreman setup. Puppet is likely right for the farm; it is wrong for five machines that one person deploys by hand. Shell scripts were never a candidate, because a script does a thing but never checks the thing.



The interface

Four surfaces sit behind one address. Each one answers a different question, for a different person. Three of them run today. The shifter’s own view is not built.

ONE DOOR, ONE ACCOUNT
THREE BUILT, ONE PLANNED
THE LIVE LANE NEEDS NO CLUSTER

00 ONE DOOR

Nobody signs in twice.

One address, on the control host

Encrypted, one account, and it leads to all four surfaces



01 FOUR SURFACES

SURFACE	WHAT IT ANSWERS	WHO OPENS IT
 Maintainer cockpit IN USE	Is the pipeline healthy? Charts and counters over the whole cluster. A shifter never asks this question.	The people who maintain the platform
 The live lane IN USE	What is arriving right now? A small React page, fed by the collectors. It asks the cluster nothing and survives an outage.	Anyone, and it holds when the cluster is down
 Discover IN USE	Anything in the archive, with filters The free-form search page of the dashboards. One box searches every source at once.	A physicist with a real question
 The shifter view COMING SOON	What is the experiment saying? It should feel close to the InfoLogger view. Familiarity is the point: nothing new to learn.	Shifters, who already know InfoLogger

The shifter view will be served through the same door. The dashboards carry two clocks: ingest time for the health panels, the archive’s own event time for the logs.

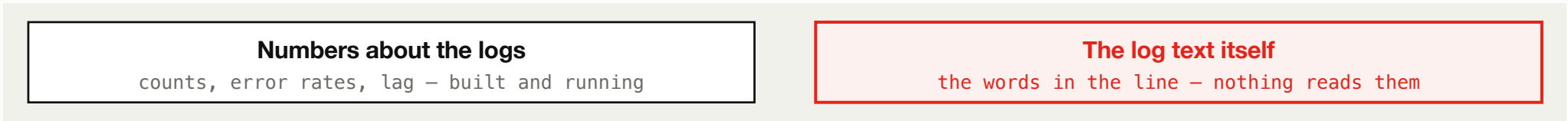
Log anomaly detection

Everything we run today watches numbers about the logs. It counts lines, errors and lag. Nothing yet reads the words in the line. That work is next, and the method is not chosen.

NOTHING HERE IS BUILT
THREE WAYS, NONE CHOSEN
PUSH UP, SCORE IN STORAGE

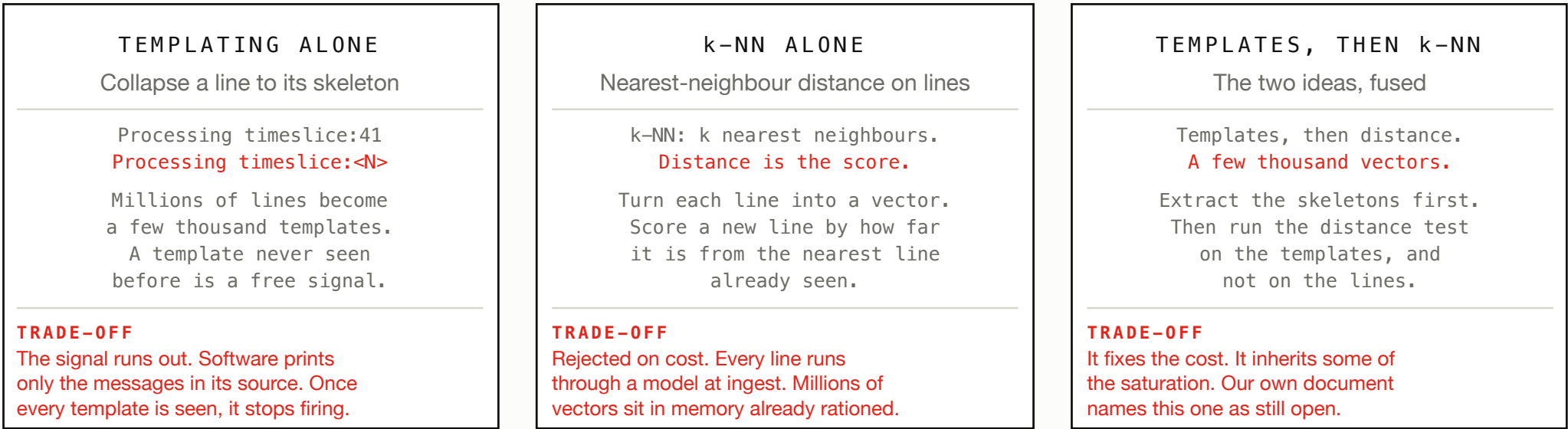
01 WHAT IS WATCHED TODAY

One of the project's next focuses.



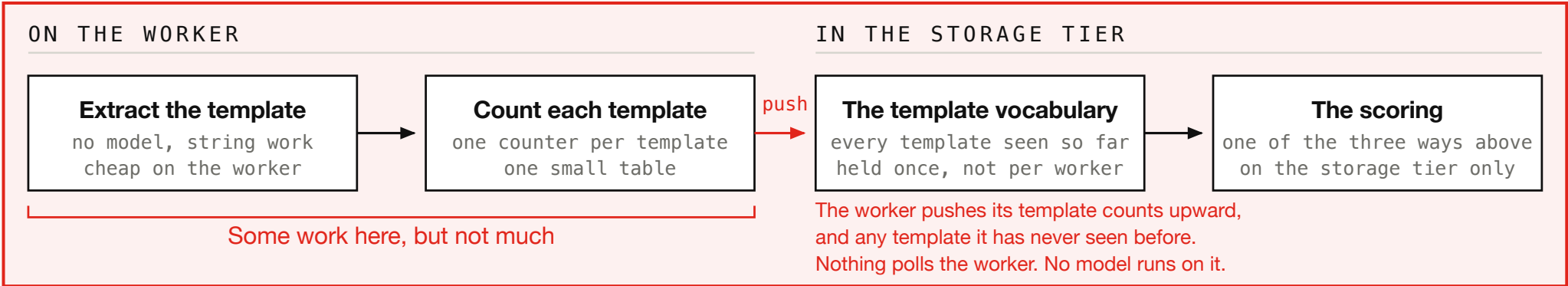
02 THREE WAYS TO READ THE TEXT

None is chosen. All three are open.



03 THE PROPOSED PIPELINE

COMING SOON



SCOPE This lane watches the numbers about the logs: counts, error rates and lag. It does not read the log text. The log text has its own lane, on the page before.

Anomaly detection on metrics

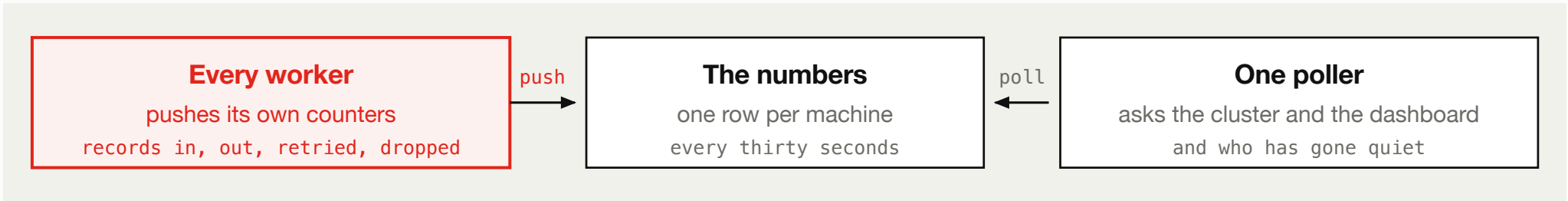
Every worker pushes its own numbers up, and nothing goes out to ask. Three lanes read those numbers. Many hits about one machine fold into one alert.

WORKERS PUSH, NOTHING SCRAPES
THREE LANES, ONE SET OF NUMBERS
ONE MACHINE, ONE ALERT

 **Feature engineering was the work.**
The detectors, the forest and the alerting all ship with OpenSearch. Choosing what to measure did not.

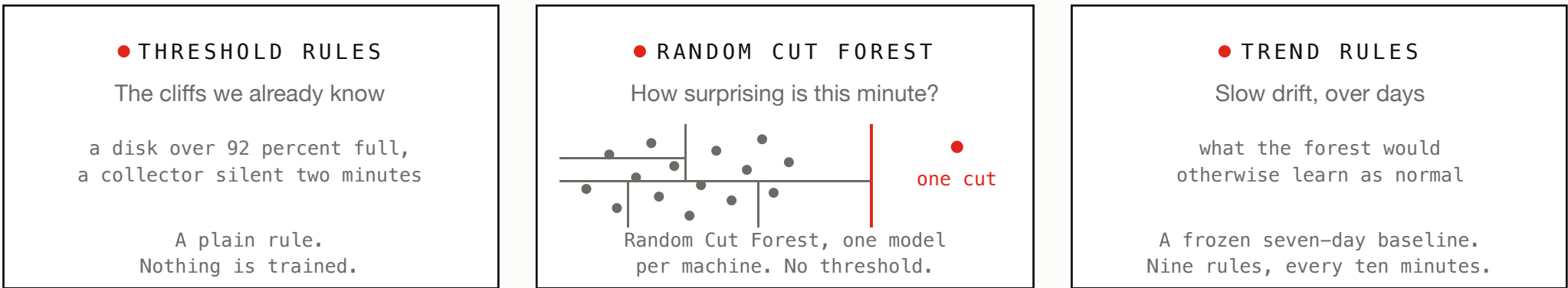
01 WHERE THE NUMBERS COME FROM

Nothing reaches into a worker to scrape it.

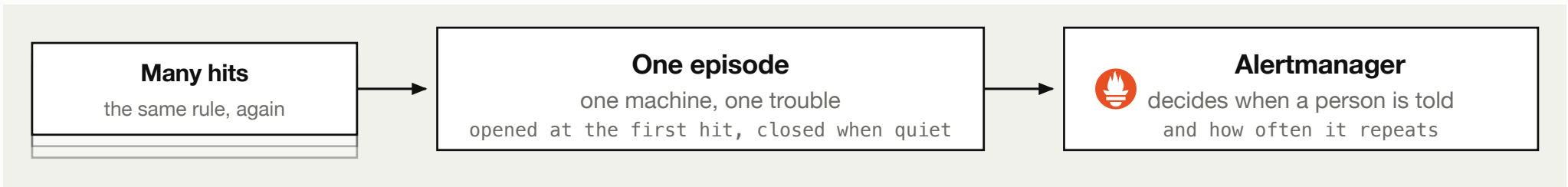


02 THREE LANES READ THOSE NUMBERS

28 saved rules and 17 detectors, all inside the cluster.



03 FROM HITS TO ONE ALERT



One broken machine is one alert, however many times the rule fires



Coming soon

Five things are missing. Each one has a reason for waiting and a price we can name. They are listed in the order we would build them.

KAFKA WAITS FOR A THIRD READER
SYSTEM LOGS NEED A REAL MACHINE
THE SHIFTER’S VIEW NEEDS EPN ACCESS

NOT BUILT YET

In the order we would build them. Each line names what holds it, and what it costs.

ITEM	WHAT IT WOULD DO	WHY IT WAITS, AND WHAT IT COSTS
01  for durability	A durable queue between collector and store - Three brokers, on the three storage machines - KRaft: Kafka keeps its own cluster state, with no separate coordination service beside it - It decouples a slow reader from a fast writer Fluent Bit writes fast, OpenSearch reads at its own pace - A slow or dead store then stops costing records	NEXT - Nothing needs it yet: two readers today, served by two collector outputs - A third reader is the trigger - Costs three brokers and a second system to run - Memory these five machines do not have
02 More log types	- Today: DDS, stdout, InfoLogger, collector health - Missing: the machine’s system journal, and the ten per-device process logs that slide 3 names - Then parse numbers out of the message body: timeframe processing time, frame size, tracks	- The archive we replay holds neither source, so this one needs a real machine - The parsers need real message shapes to test - Costs one input and one parser per machine
03 Anomaly detection on the log text	- Today we watch numbers about the logs, not the text - Slide 13 gives that gap a full page, with the three routes we are choosing between	- The choice between the routes is not made - Nothing here is built
04 The shifter’s own view	- The cockpit says whether the pipeline is healthy - A shifter asks what the experiment is saying now - A second view, close to the InfoLogger one	- It needs access to the EPN machines: we must read the running interface first - No data work; the fields are already mapped
05 Authentication inside the cluster	- One password guards the web page - Behind it, no port asks who you are - A firewall rule is the whole boundary	- Fine on five machines we own, not on the farm - Costs a certificate on every machine, an account for every job, and authentication on every call



Thank you for your attention

MODERN LOGGING PLATFORM WITH MACHINE LEARNING FOR HPC

